Управление потоком

Введение в ПИ (C++)

Control Flow

if / else

```
if (score >= 90) {
    grade = 'A';
} else if (score >= 80) {
    grade = 'B';
} else {
    grade = 'C';
}
```

Услови — выражение, приводимое к `bool`. Фигурные скобки — даже для одной строки.

Условия

```
1 ==   !=   <   <=   >   >=
2 &&   ||   !            // and / or / not
```

Частая ошибка

`if (x = 5)` — присваивание, не сравнение. Включайте `-Wall`.

switch

```
switch (cmd) {
case 1: open();  break;
case 2: save();  break;
default: help(); break;
}
```

Без `break` выполнение «проваливается» в следующий `case` *(fall-through)*.

for и while

```
for (int i = 0; i < n; ++i) {
    std::cout << i << ' ';
}

int x{};
while (std::cin >> x && x != 0) {
    sum += x;
}
```

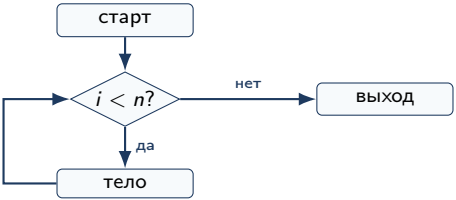
Когда какой цикл

- `for` — известны границы / счётчик
- `while` — пока условие истинно (ввод, поиск)
- `do { } while (cond);` — тело хотя бы один раз

break и continue

```
for (int i = 0; i < n; ++i) {
    if (a[i] < 0) continue;    // next iteration
    if (a[i] == 0) break;      // leave the loop
    process(a[i]);
}
```

Вложенность — рисуйте поток



Викторина 1 — сколько раз?

Викторина

Сколько раз напечатается `Hi`?

```
1 for (int i = 0; i < 3; ++i) {
2     std::cout << "Hi\n";
3 }
```

1. 2
2. 3
3. 4
4. бесконечно

Пауза — ответьте (кликер / поднимите руку / сосед), затем следующий слайд.

Ответ — Викторина 1

Ответ

2. Итерации при `i = 0, 1, 2` — три раза.

Викторина 2 — fall-through

Викторина

Что будет без `break` в `case 1`, если `cmd == 1`?

1. Выполнится только код `case 1`
2. Выполнятся `case 1` и следующие до `break`/конца
3. Ошибка компиляции всегда
4. Программа завершится

Пауза — ответьте (кликер / поднимите руку / сосед), затем следующий слайд.

Ответ — Викторина 2

Ответ

2. Это fall-through: выполнение продолжается в следующие метки.